

Master MLDM - First year - December 16, 2024

Introduction to Artificial Intelligence - Exam on Prolog

Take the things you need with you. Leave your smartphone, calculator and computer in your bag and place your bag at the front of the examination room.

Time allocated: 3h - The subject is 3 pages long - No documents allowed.

TAKE CARE: any cheating will be severely punished and will lead to a formal complaint to the disciplinary council of the university.

1 Vocabulary (2 points)

Answer the following questions, related to the Prolog language, in one or two sentences.

1. What is a Horn clause?
2. What is a fact?
3. What is a rule?
4. What is a recursive clause?
5. What is a compound term?
6. What is the Closed-World Assumption?
7. What is the Occurs Check?
8. What does DCG mean?

2 Proof tree (5 points)

Consider the Prolog program below:

- (1) $p1(A,B,C) :-$
 $q1(A,C), q2(B,C).$
- (2) $p1(A,B,C) :-$
 $q3(A), q1(B,C).$
- (3) $p1(A,B,C) :-$
 $q2(A,B), q2(B,C).$
- (4) $q3(A) :-$
 $r(A), s(B,B).$
- (5) $r(a).$ (6) $r(b).$ (7) $r(c).$
- (8) $s(1,2).$ (9) $s(2,1).$
- (10) $q1(a,b).$ (11) $q1(c,d).$
- (12) $q2(42,d).$ (13) $q2(21,d).$ (14) $q2(69,e).$
- (15) $q2(d,99).$ (16) $q2(e,97).$

1. Draw the proof tree of the resolution of the goal: $?- p1(X,Y,Z).$ and give all the solutions for this goal.
2. Suppose we put a cut between $q3(A)$ et $q1(B,C)$ in the second clause of the program. Show, on the tree you built at the previous question, which branches are pruned during the resolution of the goal: $?- p1(X,Y,Z).$ and give again all the solutions for this goal.
3. Suppose we also put a cut between $q2(A,B)$ et $q2(B,C)$ in the third clause of the program. Show, on the tree you built at the previous question, which branches are pruned during the resolution of the goal: $?- p1(X,Y,Z).$ and give again all the solutions for this goal.

3 Writing functions in Prolog (2 points)

The factorial function is defined by: $fact(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * fact(n-1) & \text{if } n > 0 \end{cases}$

The greatest common divisor function is defined by: $gcd(a,b) = \begin{cases} a & \text{if } a = b \\ gcd(a-b,b) & \text{if } a > b \\ gcd(a,b-a) & \text{if } b > a \end{cases}$

1. Write the Prolog program that defines a predicate **fact/2** where **fact(N,Res)** is true if **Res** is the result of the factorial function of **N**.
2. Write the Prolog program that defines a predicate **gcd/3** where **gcd(M,N,Res)** is true if **Res** is the result of the gcd function of **M** and **N**.

4 Unification (2 points)

For each of the goals below, say whether it is true or false. In cases where it makes sense, give the value(s) of the variable(s) that make(s) a goal true.

1. `?- 1+2 = 2+1.`
2. `?- X is Y+1, Y=4.`
3. `?- p([X|Y|[Z|T]]),Y)=p([1,2,3,4],Y).`
4. `?- p(p(p(p(p(nil,5),4),3),2),1)=p(p(X,Y),Z).`
5. `?- g(A,f(3,H),k(B,H),s(H,A,T,A))=g(3,f(B,7),k(A,T),s(T,B,7,H)).`
6. `?- [[[[[9|T],3],8],2],G]=[[A,B],D].`
7. `?- f(X,k(A,3),g(3,[1,X,3,Z]),h(Z,A))=f(2,k(1,C),g(C,[A,X,C,Z]),h(5,1)).`
8. `?- [E,[A,B],D]=[[[[[9|T],3],8],2],G].`

5 Derivative and simplification of functions (5 points)

1. Write a Prolog predicate `derivative/2` that can compute the derivative of functions of 'x' knowing that:

1. The derivative of a constant is equal to 0.
2. The derivative of x is 1.
3. If u and v are functions of 'x' then:

- (a) $(u + v)' = u' + v'$
- (b) $(u - v)' = u' - v'$
- (c) $(u \cdot v)' = u \cdot v' + v \cdot u'$
- (d) $(u/v)' = (u' \cdot v - u \cdot v')/v^2$
- (e) $(x^n)' = n \cdot x^{(n-1)}$
- (f) $\sin(u)' = u' \cdot \cos(u)$
- (g) $\cos(u)' = u' \cdot -\sin(u)$
- (h) $e(u)' = u' \cdot e(u)$
- (i) $\ln(u)' = u'/u$
- (j) $(u^v)' = v \cdot u^{(v-1)} \cdot u' + \ln(u) \cdot u^v \cdot v'$

For example if we want to process the derivative of $\sqrt{\cos(x^2)}$ which is $-x \cdot \sin(x^2) \cdot 1/\sqrt{\cos x^2}$ we can run the following goal:

```
?- derivative(cos(x^2)^0.5,R).
```

to get the result in R.

Note that there exists a built-in predicate called `atom/1` such as `atom(X)` is true if X is an atom and a built-in predicate called `number/1` such as `number(X)` is true if X is an integer or a float.

2. In fact, if you only use the above rules, you may get:

`R = 0.5*cos(x^2)^-0.5*(-sin(x^2)*(2*x^1))+ln(cos(x^2))*cos(x^2)^0.5*0` which is correct but not pleasant to read. So you have to write a second Prolog predicate, `simplify/2`, that will simplify the writing of a function. For example, if you run:

```
simplify(0.5*cos(x^2)^-0.5*(-sin(x^2)*(2*x^1))+ln(cos(x^2))*cos(x^2)^0.5*0,R).
```

you should get:

```
R = 0.5*cos(x^2)^-0.5*(-sin(x^2)*(2*x))
```

Of course, this is just one example, and you need to consider many other situations in which it is possible to simplify a function.

6 DCG (4 points)

Consider the grammar of a small programming language $\mathcal{P}$

```
program => statements
statements => statement ';'
statements => statement ';' statements
statement => variable ':= ' exp
statement => 'if' condition 'then' 'begin' statements 'end' 'else' 'begin' statements 'end'
statement => 'while' condition 'do' 'begin' statements 'end'
statement => 'read' '(' variable ') '
statement => 'write' '(' exp ') '
condition => exp operatorComp exp
exp => exp1 '+' exp
exp => exp1 '-' exp
exp => exp1
exp1 => exp2 '*' exp1
exp1 => exp2 '/' exp1
exp1 => exp2
exp2 => '(' exp ') '
exp2 => '0' exp2 => '1' exp2 => '2' exp2 => '3'
exp2 => variable
variable => an atom
operatorComp => '<'
operatorComp => '>'
```

for example, it can be used to generate/parse a program in the $\mathcal{P}$ language such as:

```
x42b := 3*(y+1) ;
while (x42b < 1) do
begin
  if (x > y)
  then begin x42b := x42b+1 ; end
  else begin x42b := 2 ; end ;
end ;
```

1. Write a DCG that can be used to say whether a program in the $\mathcal{P}$ language is syntactically correct with respect to the above grammar. Note that there exists a built-in predicate called `atom/1` such as `atom(X)` is true if `X` is an atom.
2. Provide two different ways to write the goal you have to run to say whether the program `x42b := 3*(y+1) ; y := 2*x42b` is syntactically correct or not.
3. Modify your DCG to build a tree representation of any syntactically correct program of the $\mathcal{P}$ language. For example, the tree representation of:
`if (x > y) then begin x42b := x42b+1 ; end else begin x42b := 2 ; end ;`
should be the compound term:

```
statementList(
  ifThenElse(
    comparison(>,variable(x),variable(y)),
    statementList(
      affect(
        variable(x42b),
        plus(variable(x42b),digit(1))),
      null),
    statementList(
      affect(variable(x42b),digit(2)),
      null)),
  null)
```

4. More generally, give the Prolog clause generated from the following DCG rule after loading it into the Prolog workspace: `p(A) --> q(C), [a,b], r, {s(A,C)}, [c], t(_,C)`.